Lab Manual: Binary Search Tree

In this lab you will write one C++ program from scratch. There is no skeleton file. Read the explanation, then follow the steps of the tasks. Your output must match the expected output exactly.

File name: bstlab.cpp. Task 1 and Task 2 are both written in this file, and main does the steps of Task 1 first and then the steps of Task 2.

Part 1: Binary Search Tree, Search, Insert and Traversal

What is a binary search tree?

A binary tree is a tree in which every node has at most two children, a left child and a right child. A binary search tree (BST) is a binary tree with one extra rule. For every node:

- every key in its left subtree is smaller than the key of the node, and
- every key in its right subtree is larger than the key of the node.

Duplicate keys are not allowed. Because of this rule, a search never has to look at the whole tree. At every node you decide to go left or right, and the other side of the tree is thrown away, just like in binary search on an array.

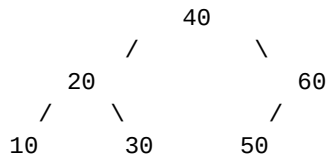
A node is stored like this. The names are the same as in the book: info is the data, llink is the pointer to the left child and rlink is the pointer to the right child. An empty child is NULL.

```
struct nodeType
{
    int info;
    nodeType *llink;
    nodeType *rlink;
};

nodeType *root = NULL;     // root points to the top node, NULL means empty tree
```

Example tree

Insert the keys 40, 20, 60, 10, 30, 50 in this order into an empty tree. The first key becomes the root. Each next key follows the left or right rule from the root down to an empty place. The result is:



Check the rule at 20: its left subtree has 10, which is smaller, and its right subtree has 30, which is larger. The same is true at every node.

Searching

Start at the root. Use a pointer current that starts at root. Repeat:

1. If current is NULL, the key is not in the tree. Stop.
2. If the key of current is the key you want, it is found. Stop.
3. If the key of current is larger than the key you want, go to the left child (current = current->llink). Otherwise go to the right child.

Search for 30 in the tree above: 30 is smaller than 40, go left. 30 is larger than 20, go right. This node is 30, found. Search for 55: 55 is larger than 40, go right. 55 is smaller than 60, go left. 55 is larger than 50, go right. The right child of 50 is NULL, so 55 is not in the tree.

Inserting

A new key always becomes a new leaf. First find the empty place for it, using the same walk as in searching. While walking you need two pointers: current, and trailCurrent which stays one step behind and always points to the parent of current. When current becomes NULL, trailCurrent is the node that will get the new node as its child.

1. Make a new node. Put the key in info and set llink and rlink to NULL.
2. If the tree is empty (root is NULL), the new node becomes root. Stop.
3. Otherwise walk down with current from the root. Before each step, copy current into trailCurrent. If the key of current is equal to the new key, it is a duplicate: print a message, delete the new node and stop. If the key of current is larger, go left, otherwise go right. Continue until current is NULL.
4. Now compare the new key with the key of trailCurrent. If the new key is smaller, the new node becomes the left child of trailCurrent. Otherwise it becomes the right child.

Insert 55 in the tree above: the walk goes 40, 60, 50 and then right of 50 is empty. trailCurrent is 50 and 55 is larger, so 55 becomes the right child of 50.

The function that inserts needs to be able to change root itself, because the first key of an empty tree becomes the new root. For this reason the parameter is written as nodeType* &root (a reference to the pointer).

```
void insertBST(nodeType* &root, int item)
```

Traversals

A traversal visits every node once. Both traversals below are written with a function that calls itself (recursion) on the left child and on the right child. A call with a NULL pointer does nothing and returns.

- Inorder: visit the left subtree, then print the node, then visit the right subtree.
- Preorder: print the node, then visit the left subtree, then visit the right subtree.

For the example tree above (before inserting 55):

```
Inorder:   10 20 30 40 50 60
Preorder:  40 20 10 30 60 50
```

Notice that the inorder traversal of any binary search tree prints the keys in sorted order. The preorder traversal shows the shape of the tree: the first key is always the root.

Lab Task 1: Insert, Search and Traversals

Write the struct nodeType as shown above and these functions.

```
void insertBST(nodeType* &root, int item)
bool searchBST(nodeType *root, int item)
void printInorder(nodeType *root)
void printPreorder(nodeType *root)
void freeTree(nodeType* &root)
```

- insertBST: inserts item as explained above. If the key is already in the tree it prints the key followed by is already in the tree, and inserts nothing.
- searchBST: returns true if the item is in the tree, otherwise false. It prints nothing.

- printInorder: prints the word Inorder: and then all keys in inorder, each with one space before it, on one line. The word must be printed only once, so you will need a second function that calls itself, and printInorder calls it.
- printPreorder: the same as printInorder, but in preorder, with the word Preorder:
- freeTree: deletes every node of the tree and sets root to NULL. Delete the left subtree and the right subtree first, and the node itself last.

Steps for main, Part 1

1. Print the line Part 1.
2. Declare root as a nodeType pointer and set it to NULL.
3. Insert these keys one by one in this order, using a loop over an array of keys: 60 50 70 30 58 80 46 77.
4. Call printInorder and then printPreorder.
5. Insert 46 again. It must be reported as a duplicate.
6. Search for 58. If searchBST returns true print the key followed by found in the tree, otherwise print the key followed by not found in the tree.
7. Do the same search and message for 77, then for 65.
8. Call freeTree.

Hint: before you run the program, draw the tree by hand from the insertion order. The first line of the preorder output must be 60, and your inorder output must be sorted.

Expected output, Part 1

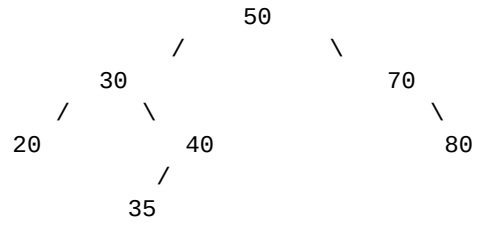
```
Part 1
Inorder: 30 46 50 58 60 70 77 80
Preorder: 60 50 30 46 58 70 80 77
46 is already in the tree
58 found in the tree
77 found in the tree
65 not found in the tree
```

Part 2: Deleting a Node

What must happen

After deleting a key, the tree must still be a binary search tree. To delete a key, first search for its node, and remember its parent (the same idea as trailCurrent in insert). If the key is not in the tree, print the key followed by is not in the tree and change nothing. Otherwise there are four cases, depending on the children of the node to be deleted.

We use this tree for the examples. It was made by inserting 50 30 70 20 40 80 35 in this order. Its preorder is 50 30 20 40 35 70 80. Each example below starts again from this tree.



Case 1: the node is a leaf (no children)

Set the pointer of the parent that points to this node to NULL and delete the node. Example: delete 20. The left pointer of 30 becomes NULL.

```
Preorder after: 50 30 40 35 70 80
```

Case 2: the node has no left child, only a right child

The parent pointer that pointed to this node now points to its right child. Then delete the node. Example: delete 70. The right pointer of 50 now points to 80.

```
Preorder after: 50 30 20 40 35 80
```

Case 3: the node has no right child, only a left child

The parent pointer that pointed to this node now points to its left child. Then delete the node. Example: delete 40. The right pointer of 30 now points to 35.

```
Preorder after: 50 30 20 35 70 80
```

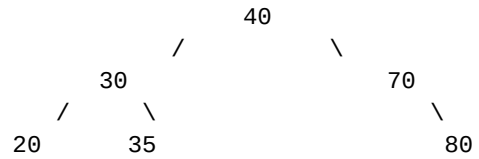
Case 4: the node has two children

We cannot give one of the pointers to the parent, because the node has two children. Instead we replace the key of the node by another key from the tree that keeps the rule true: the immediate predecessor, which is the largest key of the left subtree. Do this:

1. Go to the left child of the node to be deleted.
2. Keep going to the right child as long as there is one. You stop at the node with no right child. This is the predecessor.
3. Copy the key of the predecessor into the node that was to be deleted.
4. Now delete the predecessor node. It has no right child, so it is a Case 1 or Case 3 deletion: its parent points to its left child (or to NULL).

Example: delete 50, the root. Its left child is 30, and the right child of 30 is 40, which has no right child, so the predecessor is 40. Copy 40 into the root. Then delete the node 40: its parent 30 now points to its left child 35.

The root pointer itself stays the same node. The new tree is:




```
Preorder after: 40 30 20 35 70 80
```

Be careful in Case 4 when the left child itself has no right child. Then the predecessor is the left child, and it is the left pointer of the node to be deleted that must be changed, not a right pointer. Also remember that the node to be deleted can be the root. Then root itself must change in Cases 1, 2 and 3, so deleteBST also takes root by reference.

Lab Task 2: Delete (same file: bstlab.cpp)

Write this function. You may write extra helper functions.

```
void deleteBST(nodeType* &root, int item)
```

- deleteBST: deletes the node holding item using the four cases above. In Case 4 you must use the largest key of the left subtree. If item is not in the tree, it prints the item followed by is not in the tree and changes nothing. It prints nothing in any other case.

Steps for main, Part 2

1. Print an empty line and then the line Part 2.
2. The tree is empty after freeTree. Insert these keys one by one in this order: 60 50 70 30 53 80 75 40 57 45 77 35.
3. Call printPreorder.
4. Now delete these keys one by one, in this order: 45 30 80 50 60 99. Before each delete, print the line After deleting, the key and a colon. After each delete, call printPreorder.
5. Call printInorder.
6. Call freeTree.

Hint: draw the first tree by hand from the insertion order and delete the keys one by one on paper. 45 is a leaf (Case 1). 30 has no left child (Case 2). 80 has no right child (Case 3). 50 has two children, and its predecessor is its own left child. 60 is the root with two children, and its predecessor is deeper in the tree. 99 is not in the tree.

Expected output, Part 2

```
Part 2
Preorder: 60 50 30 40 35 45 53 57 70 80 75 77
After deleting 45:
Preorder: 60 50 30 40 35 53 57 70 80 75 77
After deleting 30:
Preorder: 60 50 40 35 53 57 70 80 75 77
After deleting 80:
Preorder: 60 50 40 35 53 57 70 75 77
After deleting 50:
Preorder: 60 40 35 53 57 70 75 77
After deleting 60:
Preorder: 57 40 35 53 70 75 77
After deleting 99:
99 is not in the tree
Preorder: 57 40 35 53 70 75 77
Inorder: 35 40 53 57 70 75 77
```